CHALLENGE 4

Two Agents, One Task

Interface interview day · Agent protocols + OAuth · Submit by 5:00 PM

THE BRIEF

Two agents owned by two different companies have to complete one task together, and the second one needs to be sure the first is really acting for a user who actually gave permission. The classic shape: Agent A is a travel assistant the user chats with; Agent B belongs to an airline and owns the flights and bookings. A can't book anything itself — it has to discover what B offers, get the user's permission, and call B on the user's behalf with a token that proves who approved what. No shared database between them; they only talk to each other.

An example flow: the user tells A 'book me on the 7pm flight to Dubai tomorrow'. A finds B, sees it can search and book, and sends the user through a consent step that returns a scoped token — say, search and book, but not cancel. A searches at B and books the 7pm flight, and B, before doing anything, checks the token really represents this user and covers this action. The booking comes back tied to the user. Later the user says 'cancel it'; the token doesn't cover cancelling, so A has to go back for permission rather than quietly trying. That's the shape — the exact flow is open to your interpretation, as long as you can clearly explain what each piece is doing and why.

For how A and B discover each other, talk, and pass that scoped permission token, use the A2A (Agent2Agent) family of protocols, or any other approach you find suitable. Pick something deliberately and be ready to defend it.

Keep both backends small — just enough to show the flow end to end. The point is the two agents talking and the permission check, not production services. airline-flights.json is sample flight data you can use or ignore in favour of your own.

IN THIS FOLDER

airline-flights.json: Sample flight data you can use or replace with your own.

WHAT YOU GET

Model key: OpenRouter key— one key reaches every model, for both agents.

WHAT TO BUILD

Two running backends: Agent A and Agent B, separate processes, talking only over the protocol.

The consent flow: The user approves once via OAuth; B receives a scoped token and verifies it before acting. End to end: 'book me on the 7pm flight' results in a booking tied to the user.

Scope enforcement: If the user asks for something the token doesn't cover (say, cancelling a booking), A goes back for consent rather than trying anyway.

WHAT TO HAND IN

The running system: Both backends, runnable from your README, completing a booking end to end.

A message trace: A short trace showing which agent sent which message and who owns each step.

Your protocol choice: Which protocol and version you picked and why.

WHAT A STRONG RESULT LOOKS LIKE

- You can explain which side owns each message and why.
- B actually checks the token's user and scope — it doesn't just trust A.
- An out-of-scope request triggers a fresh consent, not a silent attempt.
- You picked a protocol and version deliberately and can defend it.
- You can justify your model choice — why this one on cost, speed and quality, not just the biggest available.

HOW THE DAY WORKS

Deadline: Submit by 5:00 PM.

AI tools: Use any you like (Claude Code, Codex, Cursor, ChatGPT). We're judging your thinking skills, so that includes how you break the problem down, what you leave out, and how honest you are about limits.

Submission: Email one zip to noor@aiatcore.com by 5:00 PM, named <your-id>_challenge-4.zip, containing:

- Your code, with a README on how to run it (no keys or passwords in the zip; use a .env.example)
- A PPT of max 10 slides about your approach and what you built